

MyARQIA

Plan de Trabajo en Equipo · MVP

Rol	Persona
■ Tech Lead / Dev IA	Persona 1
■ Dev CAD 2D	Persona 2
■ Dev 3D / Renderizado	Persona 3
■ Dev Full-stack / Persistencia	Persona 4
■■ Arquitecta (estudiante)	Persona 5

Versión	3.0 (con prioridades + ruta crítica)
Fecha	2026-05-15
Repositorio	github.com/Jaxsdev/myarqia
Rama de integración	feature/modulo-1-topbar

1 · Visión del MVP

Cuando un usuario escribe en el chat *"genera una casa de 2 dormitorios en un lote de 7×10m"*, la app debe responder en menos de 30 segundos con un plano arquitectónico válido, renderizado limpiamente en 2D, con vista 3D opcional, que se pueda guardar y exportar como PDF.

1.1 · Criterios de aceptación del MVP

El MVP se considera entregado cuando, en una demo en vivo de máximo 10 minutos, los 10 criterios siguientes se cumplen sin tropiezos:

#	Criterio
1	El usuario abre la app, se registra, crea un proyecto.
2	Escribe en el chat: <i>"casa de 2 dormitorios con cochera"</i> .
3	En menos de 30 segundos aparece el plano en el editor 2D.
4	Las esquinas de los muros se ven limpias (sin gaps).
5	Las puertas tienen su arco de apertura visible y están sobre muros.
6	Las ventanas están solo en muros exteriores.
7	Los ambientes muestran su área en m².
8	El usuario cambia a vista 3D y puede orbitarlo con OrbitControls.
9	Guarda el proyecto, cierra el navegador, vuelve a abrir → todo está.
10	Exporta el plano como PDF.

Si los 10 puntos se cumplen → MVP entregado. Si alguno falla → es la prioridad #1 antes de cualquier nueva feature.

1.2 · Fuera de alcance del MVP (queda para v1.1+)

- Multi-piso (departamentos / casas de varias plantas).
- Mobiliario (camas, sofás, cocinas — geometría 3D de muebles).
- Exportación a DXF / IFC nativo (la base existe pero no integrada).
- Refinamiento conversacional ("hazme la cocina más grande").
- Colaboración multi-usuario en tiempo real.
- Texturas fotorealistas con HDRI environment.
- Cálculo automático de presupuesto de obra.

2 · Estado actual del prototipo (auditoría)

Auditoría hecha sobre el commit 3979508 de la rama feature/modulo-1-topbar, después del merge del PR #1 a main.

2.1 · Lo que ya funciona ■

Área	Estado actual
Editor 2D · Muros	Mitra robusta ante cualquier orientación. Snap a extremos, puntos medios, aristas e intersecciones. Cota viva al dibujar. Ortho automático 90° (Shift libera). T-junctions con gap limpio.
Motor IA (Claude)	System prompt detallado con contrato geométrico cerrado (IDs de muro, rotación en radianes, flag exterior). Validador en backend con re-intento automático si el plano viola RNE básico.
Detección de ambientes	Algoritmo de face-traversal en lib/ambientes.ts. Cuando se dibujan 4+ muros formando un cuarto cerrado, lo detecta y muestra el área en m².
Canvas 3D	Render con react-three-fiber. OrbitControls, iluminación (direccional + ambiental), ContactShadows, Environment. Componente Door3D con hoja abierta a 60°.
Persistencia	Supabase con la tabla proyectos restaurada al esquema original: uuid PK, FK a auth.users con ON DELETE CASCADE, jsonb 'datos', thumbnail, timestamps, RLS por user_id.
Autoguardado	Hook useGuardadoAutomatico que persiste cada cambio con deduplicación por hash. Captura thumbnail del canvas.

2.2 · Lo que está a medias o frágil ■■

Área	Problema concreto
Gemini API	API key bloqueada (cuota 0). Solo Claude funciona. Sin fallback.
Validador backend	Solo chequea áreas y solapamientos. No verifica que muros formen rectángulos cerrados, ni circulación, ni que puertas y ventanas estén en posiciones válidas.
Post-procesador IA	Asigna muro_id por proyección al muro más cercano (radio 50 cm). Si la IA pone una puerta lejos del muro real, se asocia mal.
Few-shot examples	Solo 2 ejemplos en el prompt (1 y 2 dorms). Faltan: estudio, 3 dorms, con cochera, con lavandería, departamento.
autoSanarPlano	Función existe en usePlanoStore.ts pero sin documentación. Hay que entenderla y testearla.
Canvas3D con mitra nueva	El 3D usa obtenerVerticesMuro de lib/muros.ts. Hay que verificar que los muros se vean alineados como en 2D después del PR #1.
Exportar PDF	Hook useExportarPDF existe pero incompleto. No produce PDF útil.
Exportar DXF	lib/exportarDXF.ts existe (264 líneas) pero sin botón ni tests.
Verificador RNE visual	useRNEStore cuenta alertas en el ContextBar pero no detalla los errores ni los resalta en el canvas.

2.3 · Lo que falta totalmente ■

Área	Detalle
Tests automáticos	Cero cobertura en frontend y backend.
CI/CD	Cada push depende de la voluntad de cada dev. Sin checks automáticos.
Deploy en producción	La app vive solo en localhost. Vercel/Netlify no configurado.
Onboarding	Sin tutorial ni guía para usuarios nuevos.
Docs API backend	Endpoints /api/chat y /api/chat-gemini sin documentar.
Catálogo de plantillas	No hay planos de referencia accesibles desde la UI.
Seguridad de keys	API keys del README inicial quedaron expuestas en historial git.

3 · Prioridades y ruta crítica

Para llegar al MVP en 4 semanas con 5 personas trabajando en paralelo, hay que priorizar despiadadamente. Usamos el modelo **MoSCoW simplificado** con tres niveles:

3.1 · Niveles de prioridad

Nivel	Significado	Acción si falta tiempo
P0 · CRÍTICO	Sin esto NO hay MVP. Bloquea la demo de aceptación.	Imposible cortar. Se entrega o se posterga el MVP.
P1 · IMPORTANTE	Mejora significativa de calidad o UX, pero la demo funciona sin esto.	Se entrega solo si hay tiempo después de cerrar todas las P0.
P2 · DESEABLE	Wow-factor o pulido. Bueno tenerlo, no esencial.	Se corta sin culpa. Pasa al backlog de v1.1.

3.2 · Matriz de prioridades del Sprint 1

Cada tarea del Sprint 1 marcada con su prioridad. La regla es: **no se empieza una P1 hasta que todas las P0 de tu rol estén avanzadas**.

Persona	P0 · Crítico	P1 · Importante	P2 · Deseable
■ P1 — IA	T1.1 Few-shot prompt T1.2 Validador fuerte	T1.3 Gemini fallback	T1.4 Doc autoSanarPlano
■ P2 — CAD 2D	T2.1 QA render IA T2.3 Etiqueta área dinámica	T2.4 Visualizador RNE	T2.2 Handles edición muros
■ P3 — 3D	T3.1 Verificar Canvas3D	T3.2 Texturas básicas T3.3 Exportar imagen 3D	T3.4 Modo tour 1ª persona
■ P4 — Persistencia	T4.1 Test E2E guardado T4.3 Botón Exportar PDF	T4.2 Thumbnails Storage T4.4 Dashboard filtros	—
■■ P5 — Arquitecta	T5.1 Biblioteca golden T5.2 Reglas RNE T5.4 QA continuo	T5.3 Matriz adyacencias	—

3.3 · Ruta crítica (dependencias entre tareas)

Estas dependencias determinan el orden de arranque. Los "→" indican *desbloquea*:

#	Cadena de dependencias	Implicación
RC1	T5.1 (Arquitecta dibuja planos golden) → T1.1 (Lead inserta como few-shot) → T5.4 (QA continuo)	Arrancar T5.1 en día 1. Sin planos golden, la IA no mejora y QA no tiene baseline.
RC2	T5.2 (Reglas RNE estructuradas) → T1.2 (Validador fuerte) + T2.4 (Visualizador RNE en canvas)	Sin RNE estructurado, el validador queda débil y el canvas no puede marcar errores.
RC3	T5.3 (Matriz adyacencias) → T1.2 (Validador puede chequear adyacencias prohibidas como baño-sala)	P5 debe entregar la matriz para que el validador chequee "baño no toca sala".
RC4	T2.1 (QA render IA) bloquea T2.2, T2.3, T2.4	Antes de pulir features 2D, hay que saber qué falla con el output real.
RC5	T3.1 (Verificar Canvas3D) bloquea T3.2 y T3.3	Si el 3D no muestra bien los muros mitrados, las texturas y exports tampoco se verán bien.

3.4 · Plan semanal sugerido

Semana	Foco principal	Hitos al final
1	Arrancar todas las P0 en paralelo. Arquitecta dibuja planos. Lead documenta validador. CAD 2D hace QA del output actual.	10 planos golden listos. Validador v1 en backend. Reporte de QA render del estado actual.
2	Cerrar P0. Iterar prompt con QA scoring. Verificar 3D. Test guardado/carga.	Demo interna del MVP: prompt → plano 2D limpio → guardar → recargar. Todos los criterios MVP 1-9 deben pasar.
3	Atacar P1: Gemini fallback, RNE visualizer, texturas 3D, thumbnails storage, dashboard filtros, exportar PDF.	Demo del MVP completo (criterios 1-10) + nice-to-haves funcionales.
4	P2 si hay tiempo. Pulido. Deploy en Vercel. Documentación final. Tutorial básico.	MVP en producción con URL pública. Documentación entregada.

4 · 5 brechas priorizadas hacia el MVP

#	Brecha	Por qué importa	Dueño	Prioridad
1	Calidad del plano que genera la IA	Todo el MVP depende de esto. Si la IA produce basura, ningún editor lo salva.	P1 + P5	P0
2	Validador robusto en backend	Sin esto, plano malo llega al frontend y el usuario lo ve.	P1 + P5	P0
3	Render limpio en 2D del output IA	Si las puertas o ventanas salen flotando, el plano se ve mal aunque la geometría sea correcta.	P2	P0
4	Guardar / reabrir sin errores	Hoy funciona post-restauración de BD, pero hay que testearlo a fondo con casos reales.	P4	P0
5	Vista 3D del plano generado	Diferenciador visual para el demo. Verificar alineación con la mitra del 2D.	P3	P1

5 · Equipo y roles

Los nombres son provisionales. Asigne cada rol según el perfil y preferencia de cada integrante. La tabla resume responsabilidades, skills mínimas y archivos principales que tocará cada uno.

Rol	Responsable de	Skills	Archivos principales
■ P1 Tech Lead IA + Backend	Coordinación. System prompt. Validador. Integración final entre backend y frontend.	Node.js, prompt engineering, geometría 2D, criterio de producto.	backend/index.js frontend/.../PanelChat.tsx
■ P2 Dev CAD 2D	Editor de muros, snap, mitra, render de aberturas, herramientas de edición (handles, trim, extend).	React, Konva, geometría 2D, atención al detalle pixel-perfect.	CanvasEditor.tsx CapaUniones.tsx ElementoMuro.tsx lib/snap.ts
■ P3 Dev 3D	Renderizado Canvas3D, materiales, iluminación, exportación de imagen/GLB, modo tour 1ª persona.	React, three.js, react-three-fiber, drei, geometría 3D.	Canvas3D.tsx lib/muros.ts
■ P4 Dev Full-stack Persistencia	Supabase, auth, dashboard, autoguardado, exportar PDF, thumbnails, sharing.	React, Supabase (SQL + Storage + RLS), jsPDF.	useGuardadoAutomatico.ts useExportarPDF.ts Dashboard.tsx
■■ P5 Arquitecta (estudiante)	Curación de planos golden, reglas RNE estructuradas, matriz de adyacencias, QA arquitectónico continuo del output de la IA.	AutoCAD o SketchUp, RNE Perú, criterio arquitectónico, planos a mano alzada con escala.	docs/planos-referencia/ docs/reglas-rne.md docs/matriz-adyacencias.md

6 · Sprint 1 — Tareas detalladas (semanas 1-2)

Objetivo del Sprint 1: cerrar las brechas P0 (#1, #2, #3, #4). Al final del sprint un prompt en el chat genera un plano que se ve correcto en 2D, se guarda y se reabre sin perder nada.

6.1 · ■ Persona 1 — Tech Lead / Dev IA

Coordina con la Arquitecta (P5) para los inputs de cada tarea. Es el puente entre criterio arquitectónico y código.

T1.1 · Ampliar few-shot del system prompt		Esfuerzo: M (4-6h)
P0 · CRÍTICO	Responsable: Persona 1 (con apoyo de P5) Archivos a tocar: backend/index.js (constante SYSTEM_PROMPT) Dependencias: T5.1 de P5 debe estar lista primero Descripción: Hoy hay 2 ejemplos completos (casa 1 dorm y 2 dorms). La IA tiende a imitar lo que ve en ejemplos; necesitamos cubrir 5+ tipologías para que generalice bien. Sub-pasos: 1. Recibir de P5 los 5 planos golden objetivo (T5.1). 2. Para cada uno, redactar el JSON completo: ambientes, muros con IDs (m1, m2, ...), puertas con muro_id + rotacion (radianes) + sentido, ventanas con muro_id + rotacion. 3. Validar manualmente que las coordenadas son múltiplos de 0.05 y que los muros forman bucles cerrados. 4. Insertar los 5 ejemplos en SYSTEM_PROMPT bajo el header ■■■ EJEMPLO COMPLETO ■■■. 5. Probar cada tipología con curl al endpoint /api/chat: que la respuesta sea JSON limpio y la IA respete el formato. 6. Iterar el prompt si la IA confunde alguna tipología. ✓ Entregable: backend/index.js actualizado + carpeta docs/qa/sprint1-prompts/ con 5 capturas del plano generado (una por tipología).	

T1.2 · Validador geométrico fuerte

Esfuerzo: L
(1-2 días)

P0 · CRÍTICO

Responsable: Persona 1

Archivos a tocar: backend/index.js (función validarPlanta)

Dependencias: T5.2 y T5.3 de P5. Bloquea a T5.4 (QA)

Descripción: El validador actual solo chequea áreas y solapamientos. Hay que cubrir las invariantes geométricas que el frontend asume.

Sub-pasos:

1. Función validarMurosCerrados(muros): cada extremo de cada muro debe coincidir con extremo de otro o estar en el bbox del lote, con tolerancia 0.05 m.
2. Función validarPuertasEnMuros(puertas, muros): cada puerta debe tener distancia perpendicular < 0.10 m a algún muro.
3. Función validarVentanasExterior(ventanas, muros): cada ventana debe estar sobre un muro con exterior:true.
4. Función validarCirculacion(ambientes, puertas): BFS desde el primer ambiente "social" debe alcanzar todos los demás.
5. Función validarAdyacenciasProhibidas(ambientes, matriz): rechaza casos como "baño pegado a sala" (usa T5.3).
6. Integrar las 5 funciones en validarPlanta() existente. Si falla, devolver mensaje claro al loop de re-intento.
7. Probar con 10 planos buenos (deben pasar) y 10 malformados manualmente (deben fallar con mensaje correcto).

✓ **Entregable:** validarPlanta() ampliada + archivo backend/tests/casos-validador.json con los 20 casos documentados.

T1.3 · Reactivar Gemini como fallback

Esfuerzo: S
(2h)

P1 · IMPORTANTE

Responsable: Persona 1

Archivos a tocar: .env (backend) + PanelChat.tsx

Descripción: Hoy Gemini está bloqueado y solo funciona Claude. Si Claude falla (rate limit, error 5xx), la app no tiene plan B.

Sub-pasos:

1. Generar nueva GEMINI_API_KEY en aistudio.google.com.
2. Actualizar backend/.env con la nueva key.
3. Probar /api/chat-gemini con un prompt sencillo vía curl.
4. En PanelChat.tsx, envolver el fetch a /api/chat en try/catch. Si falla, hacer segundo fetch a /api/chat-gemini.
5. Mostrar al usuario qué modelo respondió (badge en el mensaje).

✓ **Entregable:** Frontend con fallback automático funcionando. Captura del badge "Generado con Gemini (fallback)".

T1.4 · Documentar y testear autoSanarPlano

Esfuerzo: M
(3h)

P2 · DESEABLE

Responsable: Persona 1

Archivos a tocar: frontend/src/store/usePlanoStore.ts

Descripción: Existe una función autoSanarPlano() que se llama tras dibujar un muro, pero no hay docs de qué hace. Hay que entenderla, documentarla y testearla.

Sub-pasos:

1. Leer la función completa en usePlanoStore.ts.
2. Identificar cada operación (mover extremos cercanos, fusionar colineales, etc.).
3. Escribir JSDoc completo: @description, @example, @returns.
4. Crear scripts/tests/auto-sanacion/*.mjs con 3 casos: muros con extremos a 5 cm, muros colineales solapados, esquina imperfecta.
5. Verificar que el output de autoSanarPlano es el esperado.

✓ **Entregable:** usePlanoStore.ts con JSDoc completo + 3 archivos .mjs con tests ejecutables en Node.

6.2 · ■ Persona 2 — Dev CAD 2D

Trabajas con el output de la IA en el editor 2D. Tu misión: que el usuario pueda confiar visualmente en lo que ve y editarlo con precisión.

T2.1 · Verificar render del output IA

Esfuerzo: S
(3h)

P0 · CRÍTICO

Responsable: Persona 2 (coordina con P1 y P5)

Archivos a tocar: (solo lectura)

Descripción: Sanity check antes de cualquier feature nueva: ¿la IA actual genera planos que se ven bien?

Sub-pasos:

1. Recibir de P5 los 10 prompts representativos.
2. Ejecutar cada uno en la app local.
3. Por cada plano: capturar screenshot + anotar problemas (esquinas, posición de puertas, ventanas en muros internos, ambientes sin label).
4. Para cada problema, identificar si es (a) bug del render 2D, (b) bug del post-procesador en PanelChat, (c) calidad del output de la IA.
5. Reportar a P1 los puntos (c). Trabajar tú los puntos (a) y (b).

✓ Entregable: docs/qa-renders/sprint1.md con 10 screenshots anotados y clasificación de cada problema.

T2.2 · Handles para editar muros existentes

Esfuerzo: L
(1-2 días)

P2 · DESEABLE

Responsable: Persona 2

Archivos a tocar: ElementoMuro.tsx, CanvasEditor.tsx, usePlanoStore.ts

Descripción: Hoy solo se puede borrar y redibujar un muro. Necesitamos arrastrar los extremos para ajustar dimensiones.

Sub-pasos:

1. En ElementoMuro.tsx, cuando seleccionado=true, renderizar dos círculos pequeños en (x1,y1) y (x2,y2).
2. Cada círculo con su propio onMouseDown que dispare un modo "arrastrando extremo".
3. En CanvasEditor, en onMouseMove durante ese modo, llamar a calcularSnap(cursor, ctx) y obtener el punto snapped.
4. En onMouseUp, llamar a actualizarMuro(id, {x1:..., y1:...}) o {x2, y2} según qué extremo se movió.
5. Si el extremo se acerca a extremo de otro muro (UMBRAL = 0.10 m), snap automático.
6. Guardar el cambio en historial con saveHistory().

✓ Entregable: PR con la feature funcional + GIF de 5 s mostrando el arrastre con snap a otro muro.

T2.3 · Etiqueta de área dinámica por ambiente

Esfuerzo: S
(2h)

P0 · CRÍTICO

Responsable: Persona 2

Archivos a tocar: `CapaAmbientesDetectados.tsx`, `lib/ambientes.ts`

Descripción: Verificar que la etiqueta se actualiza al editar muros y queda en el centroide del polígono (no en bounding box).

Sub-pasos:

1. Confirmar que `CapaAmbientesDetectados` se re-renderiza cuando muros cambian (gracias a Zustand).
2. Calcular el centroide real del polígono (no el centro del bbox) — usar la fórmula del centroide de polígono.
3. Formato del label: "Sala" en la primera línea, "12.50 m²" en la segunda, `fontSize` escalado por zoom.
4. Si el ambiente es muy pequeño ($< 4 \text{ m}^2$), reducir `fontSize` proporcionalmente para que no se desborde.

✓ **Entregable:** PR con el fix + screenshot de un plano con 6 ambientes etiquetados correctamente.

T2.4 · Visualizador de errores RNE en canvas

Esfuerzo: M
(4h)

P1 · IMPORTANTE

Responsable: Persona 2 (coordina con P5)

Archivos a tocar: `userNEStore.ts`, nuevo componente `CapaAlertasRNE.tsx`

Dependencias: T5.2 de P5

Descripción: Hoy hay un contador en el `ContextBar` pero el usuario no sabe qué ambiente viola la norma ni por qué.

Sub-pasos:

1. Recibir de P5 las reglas RNE estructuradas (T5.2).
2. En `userNEStore`, agregar `alertasPorAmbiente`: `Map<id, {regla, msg}[]>`.
3. Función `verificarRNE(ambientes, muros, puertas, ventanas)` que aplica las reglas y popula el `Map`.
4. Llamar a `verificarRNE` cada vez que cambian los muros (con `debounce 500 ms`).
5. Componente `CapaAlertasRNE`: dibuja círculo rojo punteado alrededor de cada ambiente con alertas.
6. Al hacer hover sobre el círculo, mostrar tooltip con la lista de errores.

✓ **Entregable:** PR con feature + screenshot de un plano que viola 3 reglas RNE.

6.3 · ■ Persona 3 — Dev 3D / Renderizado

Eres dueño del Canvas3D y de cómo se ve el plano en tres dimensiones. Tu trabajo determina cuánto impacto visual tiene el demo del MVP.

T3.1 · Verificar Canvas3D con la mitra del merge

Esfuerzo: S
(2-3h)

P0 · CRÍTICO

Responsable: Persona 3

Archivos a tocar: Canvas3D.tsx, lib/muros.ts (obtenerVerticesMuro)

Descripción: El merge del PR #1 cambió la geometría de los muros en 2D. Hay que verificar que el 3D usa la misma mitra y no muestra desalineaciones.

Sub-pasos:

1. Generar 5 planos con la IA en 2D (los mismos de T2.1).
2. Switchear a vista 3D para cada uno.
3. Verificar: ¿los muros aparecen? ¿están alineados como en 2D? ¿las puertas tienen su arco abierto? ¿las ventanas son huecos visibles?
4. Si hay desalineación, comparar la lógica de obtenerVerticesMuro (lib/muros.ts) con calcularCarasMuro + ajustarPoligonoMuro (CapaUniones.tsx).
5. Idealmente: extraer la lógica de mitra a un módulo común lib/geometriaMuros.ts y consumirla desde ambas (2D y 3D).
6. Reportar findings y commitear los fixes.

✓ **Entregable:** docs/qa-renders/3d-sprint1.md con comparativa 2D vs 3D para 5 planos.

T3.2 · Texturas básicas para muros y piso

Esfuerzo: M
(4-6h)

P1 · IMPORTANTE

Responsable: Persona 3

Archivos a tocar: Canvas3D.tsx, nueva carpeta public/textures/

Descripción: Hoy los muros son blanco plano. Para el demo necesitamos al menos 3 opciones de material visualmente diferenciadas.

Sub-pasos:

1. Descargar de polyhaven.com 3 texturas seamless (Diffuse + Normal + Roughness): muro pintado blanco, ladrillo expuesto, drywall.
2. Optimizar a 1024×1024 .jpg (< 200 KB c/u).
3. Colocar en public/textures/{muro-pintado,ladrillo,drywall}/.
4. Crear hook useTexturaMuro(material) que devuelva las 3 texturas vía useTexture de drei.
5. En Canvas3D, aplicar al meshStandardMaterial de cada muro según muro.material.
6. Agregar selector en ContextBar para elegir material.

✓ **Entregable:** PR con feature + 3 screenshots (uno por material).

T3.3 · Botón "Exportar imagen 3D"**Esfuerzo: S**
(2h)**P1 · IMPORTANTE****Responsable:** Persona 3**Archivos a tocar:** Canvas3D.tsx, TopBar.tsx**Descripción:** Para que el usuario comparta el render fotográfico de su plano.**Sub-pasos:**

1. Obtener referencia al renderer de three.js vía useThree.
2. Función exportarVista3D(): llamar gl.toDataURL('image/png') con render a tamaño 1920×1080.
3. Crear Blob desde dataURL y descargar con un link <a download>.
4. Botón en TopBar "Exportar 3D" que dispare la función.

✓ Entregable: PR con feature + PNG de muestra de 1920×1080.**T3.4 · Modo tour: cámara primera persona****Esfuerzo: L** (1
día)**P2 · DESEABLE****Responsable:** Persona 3**Archivos a tocar:** Canvas3D.tsx**Descripción:** Para que el usuario "camine" dentro de su plano (killer demo).**Sub-pasos:**

1. Importar PointerLockControls de @react-three/drei.
2. Agregar toggle "Modo tour" en el TopBar (solo visible en vista 3D).
3. Cuando activo: cambiar OrbitControls por PointerLockControls. Click activa el lock.
4. Agregar movimiento WASD vía useFrame (incrementa position según direction del controls).
5. Velocidad ajustable (default 0.05 unidades/frame).
6. ESC sale del modo tour y vuelve a OrbitControls.
7. Mostrar un crosshair al centro de la pantalla mientras está activo.

✓ Entregable: PR con feature + video corto del recorrido por una casa.

6.4 · ■ Persona 4 — Dev Full-stack / Persistencia

Te encargas de que el usuario nunca pierda su trabajo y de que la experiencia entre dashboard y editor sea fluida.

T4.1 · Test E2E de guardado y carga

Esfuerzo: S
(3h)

P0 · CRÍTICO

Responsable: Persona 4

Archivos a tocar: (solo verificación)

Descripción: Después de la restauración de BD, hay que validar a fondo que todo el flujo de persistencia funciona.

Sub-pasos:

1. Documentar 5 escenarios: (a) Crear → dibujar 10 muros → recargar → verificar todo. (b) Crear → IA genera plano → guardar → cerrar sesión → reabrir. (c) Editar proyecto existente → cambiar ambiente → autoguardar. (d) Borrar proyecto desde Dashboard → confirmar que no aparece. (e) Verificar thumbnail aparece en dashboard.
2. Ejecutar cada escenario manualmente.
3. Para cada uno: capturar pantalla del antes y después + verificar el row en Supabase Studio.
4. Reportar bugs encontrados como issues de GitHub.

✓ **Entregable:** docs/tests-e2e/sprint1.md con 5 escenarios documentados.

T4.2 · Thumbnails en Supabase Storage

Esfuerzo: M
(4-6h)

P1 · IMPORTANTE

Responsable: Persona 4

Archivos a tocar: useGuardadoAutomatico.ts, lib/supabase.ts

Descripción: Hoy los thumbnails se guardan como dataURL base64 en la columna thumbnail. Eso pesa mucho y satura la BD.

Sub-pasos:

1. Crear bucket "thumbnails" en Supabase Studio.
2. Configurar RLS del bucket: user puede subir/leer/borrar solo objetos dentro de su carpeta {user_id}/.
3. En useGuardadoAutomatico.capturarThumbnail: convertir dataURL a Blob (fetch(dataURL).then(r => r.blob())), subir vía supabase.storage.from("thumbnails").upload(...), path {user.id}/{project.id}.png, obtener URL pública con getPublicUrl, guardar solo la URL en proyectos.thumbnail.
4. Migrar thumbnails existentes (los nuevos usuarios no los tendrán, pero por si acaso).
5. Actualizar Dashboard para mostrar el .

✓ **Entregable:** PR con feature + verificación: tabla proyectos.thumbnail contiene URLs cortas, no dataURLs.

T4.3 · Botón Exportar PDF funcional

Esfuerzo: L (1 día)

P0 · CRÍTICO

Responsable: Persona 4

Archivos a tocar: useExportarPDF.ts, TopBar.tsx

Descripción: El hook existe pero está incompleto. Necesitamos un PDF presentable. Es el criterio #10 del MVP.

Sub-pasos:

1. Implementar useExportarPDF completo con jsPDF: hoja A4 horizontal, membrete (logo MyARQIA + nombre + fecha + escala 1:50 o 1:100), imagen del canvas 2D vía canvas.toDataURL → doc.addImage, tabla de ambientes con áreas, pie con firma del usuario y RUC/CIP si aplica.
2. Botón "Exportar PDF" en TopBar (icono download).
3. Manejar el loading state (botón deshabilitado mientras genera).
4. Probar con 3 proyectos distintos (estudio, casa 2 dorms, casa con cochera).

✓ **Entregable:** PR con feature + 3 PDFs de muestra adjuntos.

T4.4 · Dashboard con filtros y orden

Esfuerzo: M (4h)

P1 · IMPORTANTE

Responsable: Persona 4

Archivos a tocar: Dashboard.tsx

Descripción: Hoy el dashboard solo lista todo en orden fijo. Con 50+ proyectos se vuelve inusable.

Sub-pasos:

1. Input de búsqueda que filtra por proyecto.nombre (debounce 300 ms).
 2. Selector de orden: "Más reciente", "Más antiguo", "A-Z", "Z-A".
 3. Filtro por mes/año (dropdown).
 4. Vista alternativa: grilla (cards con thumbnail) vs lista (tabla compacta).
 5. Persistir las preferencias en localStorage.
- ✓ **Entregable:** PR con feature + screenshot mostrando 30 proyectos filtrados.

6.5 · ■■ Persona 5 — Arquitecta (estudiante)

Tu rol es el más importante del MVP en términos de calidad. La IA produce JSON, pero TÚ defines qué hace que un plano sea bueno arquitectónicamente. Sin tu input, la IA generará planos geoméricamente correctos pero mal distribuidos.

T5.1 · Biblioteca de planos golden (10-15 planos)		Esfuerzo: L (3-5 días)
P0 · CRÍTICO Responsable: Persona 5 Archivos a tocar: docs/planos-referencia/*.png + .md por plano Descripción: Crear referencias visuales y documentadas que sirvan de entrenamiento few-shot para la IA. Sub-pasos: 1. Producir 10-15 planos en formato dibujo a mano alzada con escala (foto), o en AutoCAD/SketchUp, exportados a PNG. 2. Tipologías a cubrir (mínimo): (1) Estudio 30 m² lote 5×6m. (2) Casa 1 dorm lote 6×8m. (3) Casa 2 dorms lote 7×10m. (4) Casa 3 dorms lote 8×12m. (5) Casa con cochera. (6) Casa con lavandería. (7) Casa con depósito. (8) Departamento tipo edificio. (9) Vivienda mínima RNE. (10) Casa con escalera interior (preview multi-piso). 3. Para cada plano, crear un .md con: resumen (área techada, ambientes, orientación), imagen, tabla de ambientes (posición, dimensiones, área, notas), lista de adyacencias clave, decisiones de diseño explicadas (1-2 líneas c/u), cumplimiento RNE (área, lados, ventanas). 4. Subir todo a docs/planos-referencia/ y crear un INDEX.md. ✓ Entregable: Carpeta docs/planos-referencia/ con 10+ planos completos y documentados.		
T5.2 · Documento de Reglas RNE estructurado		Esfuerzo: M (1-2 días)
P0 · CRÍTICO Responsable: Persona 5 Archivos a tocar: docs/reglas-rne.md Descripción: Reescribir las reglas RNE Perú en formato tabla, para que la IA y el validador del backend las consuman directamente. Sub-pasos: 1. Investigar y consultar el RNE (Reglamento Nacional de Edificaciones del Perú), normas A.010 (vivienda) y A.020 (habilitaciones). 2. Tabla en markdown con columnas: Ambiente, Área mín (m²), Lado mín (m), Ventana exterior obligatoria, Puerta ancho (m), Altura libre mín (m), Notas RNE (Art./Norma). 3. Cubrir todos los ambientes habituales: sala, comedor, cocina, dormitorio simple, dormitorio principal, baño completo, baño visita, lavandería, pasillo, hall, cochera, escalera, terraza, depósito. 4. Agregar sección sobre "reglas geométricas" (separación mínima entre ventanas, anchos mínimos de puerta por tipo, etc.). 5. Revisar con el Tech Lead que la tabla es procesable por la IA. ✓ Entregable: docs/reglas-rne.md con tabla completa y citado de artículos.		

T5.3 · Matriz de adyacencias ideales

Esfuerzo: S
(4-6h)

P1 · IMPORTANTE

Responsable: Persona 5

Archivos a tocar: docs/matriz-adyacencias.md

Descripción: Tabla 2D que dice qué ambiente debe estar al lado de qué otro. La IA usará esto para evitar absurdos como "baño al lado de sala".

Sub-pasos:

- 1. Lista de ambientes a considerar (mismos de T5.2).
- 2. Tabla simétrica donde cada celda tiene un nivel: ✓✓✓ ideal (deben compartir muro), ✓✓ muy recomendable (deben estar cerca), ✓ aceptable, ~ indiferente, ✗ evitar.
- 3. Documentar el racional debajo de la tabla (ej: "cocina ↔ comedor es ideal porque facilita el servicio").
- 4. Casos especiales: el baño NUNCA debe dar directamente a la sala o al comedor sin pasillo intermedio.

✓ Entregable: docs/matriz-adyacencias.md con tabla 12×12 + racional.

T5.4 · QA continuo de salidas de la IA

Esfuerzo: M
(5-8h totales
en el sprint)

P0 · CRÍTICO

Responsable: Persona 5

Archivos a tocar: docs/qa-scoring/sprint1.xlsx o .md

Dependencias: T5.2 y T5.3 listas para definir criterios

Descripción: Cada vez que P1 modifica el system prompt, generar 5 planos en tipologías distintas y calificarlos. Reportar al equipo qué falla.

Sub-pasos:

- 1. Definir 5 prompts de prueba: "casa 1 dorm 6×8", "casa 2 dorms con cochera", "estudio 30 m²", "casa 3 dorms", "depto 2 dorms".
- 2. Por cada plano generado, llenar tabla de scoring con 6 criterios ponderados: Distribución zonal (25%), Adyacencias (20%), Circulación (20%), Cumple RNE (15%), Ubicación y sentido de puertas (10%), Ventanas en muros exteriores (10%).
- 3. Cada criterio se califica 1-5. Score total ponderado.
- 4. Reportar a P1: "el plano X falla en circulación porque la única forma de llegar al baño es por el dormitorio".
- 5. Iterar al menos 3 veces durante el sprint (después de cada ajuste del prompt).

✓ Entregable: docs/qa-scoring/sprint1.* con ≥30 planos calificados (5 × 6 iteraciones).

7 · Sprint 2 — Pulido y deploy (semanas 3-4)

Objetivo del Sprint 2: pulir el MVP y desplegarlo en producción para el demo. Aquí las tareas son más cortas y muchas dependen del Sprint 1.

Person a	Tarea	Esfuerzo	Prioridad
P1	Endpoint /api/refinar para ajustar plano con prompt incremental ("haz la cocina más grande").	L	P2
P1	Logging estructurado en backend: prompts entrantes, respuestas IA, validation errors.	S	P1
P2	Cotas RNE automáticas: dimensionar cada muro al guardar.	M	P1
P2	Herramienta trim: cortar un muro al chocar con otro.	M	P2
P3	Render con HDRI environment (más fotorealista).	M	P2
P3	Exportar GLB del plano para Blender/Unity.	S	P2
P4	Compartir proyecto por link público (solo lectura).	M	P1
P4	Deploy en Vercel + dominio + variables de entorno.	M	P0
P5	Biblioteca de mobiliario tipo (cama, sofá, mesa, inodoro) con dimensiones para v1.1.	L	P2
P5	Glosario arquitectónico para el chat (vano, alféizar, antepecho — qué significan).	S	P2

8 · Definition of Done

Una tarea está terminada cuando se cumplen los 7 puntos:

#	Criterio
1	Compila limpio — <code>npm run build</code> y <code>tsc -b</code> exit code 0.
2	Probado manualmente — al menos 3 casos de uso ejercitados.
3	Cubre 2D y 3D — si toca el editor, se verificó en ambos.
4	Cubre ambas IAs — si toca el backend, se probó con Claude y Gemini (cuando esté reactivado).
5	Commit claro — en rama <code>feat/<persona>/<tarea></code> con mensaje descriptivo en español.
6	Pull Request — abierto contra <code>feature/modulo-1-topbar</code> con descripción + screenshots/GIFs.
7	Aprobación — al menos un  de revisión de otro dev.

9 · Git flow

9.1 · Branches

Rama	Propósito
main	Producción / demo. Solo recibe merges de PR aprobados.
feature/modulo-1-topbar	Rama de integración activa. Aquí mergean los devs sus features.
feat/<persona>/<tarea>	Cada tarea su rama. Ej: feat/sofia/biblioteca-planos.

9.2 · Workflow

Paso	Comando o acción
1	<code>git checkout feature/modulo-1-topbar && git pull</code>
2	<code>git checkout -b feat/<tunombre>/<tarea></code>
3	Trabajar. Commits frecuentes con mensajes claros en español.
4	<code>git push -u origin feat/<tunombre>/<tarea></code>
5	<code>gh pr create --base feature/modulo-1-topbar</code> (o desde la web).
6	Esperar revisión de otro dev. Si hay cambios pedidos, <code>git push</code> los actualiza.
7	Merge (squash o merge commit según el contenido) y borrar la rama.

9.3 · Convención de commits

En español, imperativo, conciso. Prefijos: feat:, fix:, docs:, chore:, refactor:, test:.

■ Buenos	■ Malos
feat(editor): handles para arrastrar extremos de muros	cambios al editor
fix(ia): puerta sin muro_id falla silenciosamente	fix
docs: añadir matriz de adyacencias	actualizar archivo
refactor(snap): extraer lógica de ortho a función separada	limpieza

10 · Reuniones recomendadas

Reunión	Frecuencia	Duración	Participantes
Daily sync	Diario	15 min	Todos
Demo de planos generados por la IA	2× por semana	30 min	P1 + P5 + quien quiera
Revisión visual de render (2D y 3D)	Semanal	45 min	P2 + P3 + P5
Retrospectiva de sprint	Cada 2 semanas	1 hora	Todos

11 · Cheatsheet de comandos

11.1 · Levantar la app local

```
cd backend && npm install && npm run dev # puerto 3001
cd frontend && npm install && npm run dev # puerto 5173
```

11.2 · Compilar para producción

```
cd frontend && npm run build
```

11.3 · Crear Pull Request rápido

```
gh pr create --base feature/modulo-1-topbar \
  --title "feat(...): ..." \
  --body "..."
```

11.4 · Probar la IA por curl

```
curl -X POST http://localhost:3001/api/chat \
  -H "Content-Type: application/json" \
  -d '{"mensajes":[{"role":"user","content":"casa 2 dorms"}]}'
```

12 · Variables de entorno

■■ Importante: las API keys que estaban hardcodedas en el README inicial deben rotarse — quedaron expuestas en el historial git público. Cada dev genera su propio par.

12.1 · backend/.env

```
PORT=3001
ANTHROPIC_API_KEY=sk-ant-...
GEMINI_API_KEY=...
```

12.2 · frontend/.env.local

```
VITE_SUPABASE_URL=https://cpapnhdtsifblkpbfxwl.supabase.co
VITE_SUPABASE_ANON_KEY=eyJhbG...
VITE_BACKEND_URL=http://localhost:3001
```

13 · Demo de aceptación del MVP

El MVP se considera entregado cuando, en una demo en vivo de máximo 10 minutos, los 10 puntos siguientes se cumplen sin tropiezos:

1. El usuario abre la app, se registra, crea un proyecto.
2. Escribe en el chat: "casa de 2 dormitorios con cochera".
3. En menos de 30 segundos aparece el plano en el editor 2D.
4. Las esquinas de los muros se ven limpias (sin gaps).
5. Las puertas tienen su arco de apertura visible y están sobre muros.
6. Las ventanas están solo en muros exteriores.
7. Los ambientes muestran su área en m².
8. El usuario cambia a vista 3D y puede orbitarlo con OrbitControls.
9. Guarda el proyecto, cierra el navegador, vuelve a abrir → todo está.
10. Exporta el plano como PDF.

Si los 10 se cumplen → MVP entregado. Felicitaciones al equipo y pasamos al backlog de v1.1.

Este documento es un contrato vivo. Revísenlo cada sprint planning y ajústelo según lo aprendido. La meta no es seguirlo al pie de la letra: es tener claridad de qué hace cada uno y cómo se mide el éxito.